University of Pisa
Master's Degree in Artificial Intelligence and Data Engineering
Distributed Systems and Middleware Technologies

VoltShare

Project Documentation

A distributed coordination system for electric-vehicle charging stations

Group members:

Andrea Innocenti
Lorenzo Barci

Academic Year 2025/2026

Contents

1 Introduction

VoltShare coordinates a network of electric-vehicle charging stations. Drivers find a station, reserve a connector, plug in and are billed; the operator sees the network from a back office. Underneath, the system decides three things: which vehicle gets which connector, how the limited electrical capacity of a site is shared among the cars charging at it, and what happens to both when a node fails.

This domain represents a real contention scenario: different drivers could request the same connector at the same time. This problem cannot be resolved in an easy way by a central authority that assigns arbitrarly the driver to a resource, because they have their own destination and can ignore any suggestion. The system can only *arbitrate the requests that drivers make.*

1.1 The problems this project solves

Four of them:

- **Exclusive access to a physical resource.** Several drivers ask for the same connector at the same instant, and the outlet must end up promised to one of them.
- **One reservation per vehicle across the whole network.** This is the hard one. Two stations can each be locally correct and still break the guarantee together, so it needs an authority, which then needs replication, election and a quorum to survive its own failure.
- **Sharing a divisible resource.** Tipically, in a real word scenario, the total site capacity is lower than the sum of what its connectors are rated for, so the available power is apportioned among active sessions and renegotiated whenever a car arrives, leaves or fills up.
- **Failures that must not take resources with them.** Nodes crash and links break, and from the outside the two are indistinguishable. A reservation held by a node that has stopped answering must be released without waiting for it to return.

Section 5 states all seven problems precisely, section 7 gives the mechanism chosen for each, and section 8 covers the failures.

The outlet, its contactor, its meter and the car are an emulator speaking a reduced set of OCPP 1.6-J messages (see section 9.3), so the boundary falls where a real interface already exists (section 4.1.5). Seven containers on one host run everything else (section 4.2).

Measurements come from running deployments and where something was not measured, the text says so, and the limits are declared in section 8.8.

2 Requirements Specification

Five parties take part. The names in table 1 are used in this exact sense for the rest of the document.

2.1 Functional requirements

Account and vehicle. Registration and login. A vehicle profile carries battery capacity and maximum charging power. Both are inputs to the power allocation.

Station discovery. A list of stations. Each one shows its connectors, its site power and its tariff. The figures follow the live state of the cluster.

Reservation. A driver reserves a specific connector under a **lease**: a fixed window in which to arrive. When it expires the connector is released. No operator acts, and the client does

Table 1: Actors. The charge point is a peer of the system; the browser is a client of it.

Actor	Role
Driver	a registered user with one vehicle profile; searches, reserves, charges, is billed
Station	a site with N connectors and a maximum site power, run by one node
Connector	one outlet: the exclusive resource of the system
Charge point	the equipment that governs a connector. It reports what is connected and how much is drawn, and it obeys power limits. It has a channel of its own, separate from the driver's
Operator	the back office: registry, history, billing. Read-only in this scope

not have to cooperate. The driver may cancel earlier. *A vehicle holds at most one active reservation across the whole network*, even under concurrent requests to two stations. Repeated no-shows suspend the right to reserve.

Charging session. A session starts when the charge point reports that a vehicle has connected. The station authorises it against the connector's reservation. A walk-in on a free connector is authorised against the driver's account instead. While the session runs, the driver's page receives power, energy, state of charge and an estimated completion time by push. The session ends on the driver's command or with a full battery. A vehicle that stays connected after that is flagged as overstaying and charged per minute.

Power allocation. The site power is lower than the sum of the connectors' ratings, so it is split among the active sessions. The split is recomputed on every arrival, every departure and on a periodic tick. Below a minimum viable power a session is suspended rather than starved.

Billing and history. Cost is computed after a session ends, from energy at the station tariff plus the overstay charge. A missed reservation is never billed. It costs the right to reserve.

Notifications. Reservation about to expire, expired, claim revoked, charge complete, overstay started, session interrupted. They are delivered live on the open page and stored for the next visit.

2.2 Non-functional requirements

Exclusive use. A connector serves at most one session at a time, and a vehicle holds at most one reservation network-wide. Neither invariant may be broken by concurrent requests, by a node failure or by a network partition (sections 7.1, 7.2 and 7.4).

No resource is lost to a failure. A crashed station must leave no connector reserved for ever. It must leave no driver holding a phantom reservation that blocks them elsewhere. Every hold expires on its own (sections 7.3 and 8.3).

Station autonomy. A station keeps serving the vehicles already connected to it while the rest of the network is unreachable (measured in section 8.3).

Real time. State changes reach connected clients by server push. Nothing in the system polls (sections 7.6 and 9.5).

Horizontal growth. Adding a station means adding a node. It announces itself to the coordinator at start-up, and the directory follows (section 9.4).

Out of scope from the start: real payment processing, full OCPP compliance, maps and routing, a native mobile application, and plug-and-charge authentication (ISO 15118).

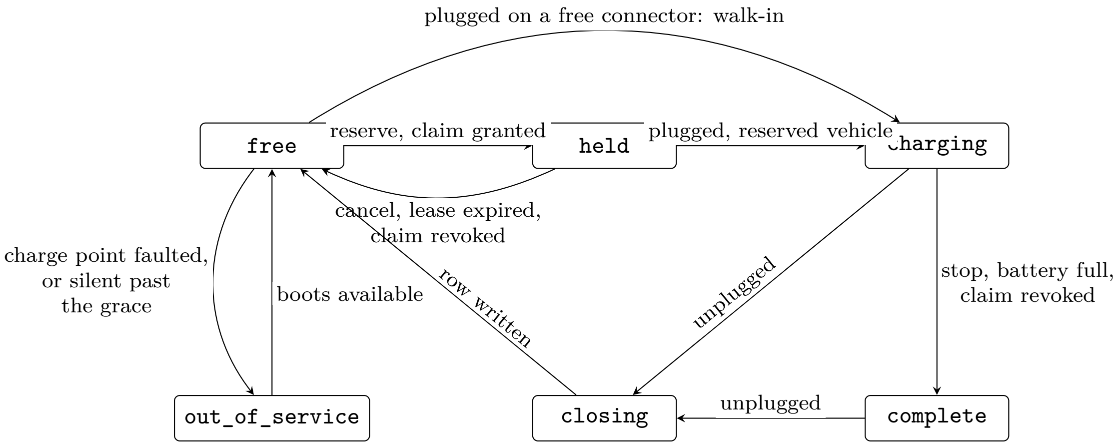

Figure 1: The connector's states, one `gen_statem` per connector. `overstay` is `complete` past the grace and `suspended` is `charging` at a limit of zero; both are derived and reported to drivers as states of their own. The fault arc leaves every state and is drawn once; `closing` returns to `free` once the session row is written and the claim released.

3 Domain Rules

The rules the system enforces, in the order a driver meets them. Every number is configuration, and table 2 gives the defaults beside the shorter demonstration values. No clock is scaled.

Reservation and lease. A reservation covers one connector at one station and holds it for `LEASE_SECONDS`, 15 minutes by default. If nobody arrives, the connector frees itself, with no operator and no client involved. Two minutes before the end the driver is warned. A reservation is granted only after the coordinator has issued a **claim** for the vehicle (section 7.2), which is what enforces the network-wide rule. The claim outlives the reservation by `CLAIM_GRACE_SECONDS`, so it never expires underneath a live one.

No-show and suspension. Arriving resets the counter. After `PENALTY_NO_SHOWS` consecutive expired reservations (two) the account loses the right to reserve for `PENALTY_DAYS` (one). The suspension is on the account, so a second vehicle does not escape it, and it suspends reserving only: a suspended driver can still walk in and charge on a free connector. The coordinator enforces it at claim time, on a path the reservation already takes. A missed reservation is never billed.

Authorisation. A session starts when the charge point reports a vehicle, and only then. On a held connector the vehicle has to be the one that reserved; any other is refused, and the reservation stays. On a free connector any registered vehicle starts a walk-in session, with no claim involved. Identification is by vehicle, and the station resolves the account from it in MySQL. An isolated station cannot make that lookup, and starts no session (section 8.3).

End of charge, grace, overstay. A session ends when the driver stops it, when the battery reaches 100%, or when the coordinator revokes the claim. The vehicle is still connected, so the connector stays occupied. For `OVERSTAY_GRACE_SECONDS` (five minutes) that costs nothing. After the grace the connector is `overstay`, and every started minute is charged at the station's overstay rate. No software moves a car whose driver does not come back, so the charge is the only lever left.

Power. A site has a budget lower than the sum of its connectors' ratings, and it is split among the active sessions by *max-min fair share with hand-back* (section 7.5). Every session starts from an equal share; one that cannot absorb its share hands the surplus back, and the surplus is split again. The split is recomputed on every arrival, every departure and every tick, and each charge point is told its limit. A session whose share would fall below `MIN_CHARGE_KW` (6 kW) is `suspended` instead: it stays alive, draws nothing, and the driver is told.

Tariff and billing. Each station has a tariff in cents per kWh and an overstay rate in cents per minute, and cost is energy at the tariff plus the started overstay minutes. The back office computes it on a periodic sweep, after the session row has been written. Billing is never a contended resource.

Table 2: The parameters behind the rules. All are environment variables read at start-up; the demonstration profile is `deploy/.env.demo`.

Parameter	Default	Demo	Meaning
`LEASE_SECONDS`	900	90	reservation lease
`CLAIM_GRACE_SECONDS`	60	30	how long a claim outlives its reservation
`PENALTY_NO_SHOWS`	2	2	consecutive no-shows that suspend the account
`PENALTY_DAYS`	1	1	length of the suspension
`OVERSTAY_GRACE_SECONDS`	300	20	free time after the end of the charge
`tariff_cents_min_overstay`	50	50	overstay rate, per station, in the database
`MIN_CHARGE_KW`	6	6	below this a session is suspended
`SITE_POWER_KW`	350 / 180	200 / 180	site budget of the two stations
`BILLING_SWEEP_SECONDS`	60	10	how often closed sessions are priced

4 System Architecture

4.1 Design decisions

We present five decisions that shaped the system and some alternatives are discarded and the reason. The sixth and largest, how the network-wide uniqueness of a reservation is enforced, belongs with the problem it answers and is in section 7.2.

4.1.1 The case for middleware

Three of the edges in this system could in principle be written directly on TCP. None of them is, and the reasons differ per edge, which is why we treat them separately instead of adopting one communication technology everywhere.

Between Erlang nodes: distributed Erlang. The stations and the coordinators exchange claims, renewals, announcements and node-failure notifications. On raw sockets each of these would need a framing format, a request-response correlation scheme, a reconnection policy and a way of noticing that a peer has died. Distributed Erlang provides all four: messages are typed terms, a process can be addressed as `{Name, Node}` without knowing an address, and `monitor_node/2` turns a dead peer into a message that arrives in a mailbox like any other. Section 8.2 shows where that last mechanism was not enough on its own and what we added.

Between Java and Erlang: JInterface. The back office has to receive the live station list from the coordinator and to push suspensions to it. The two runtimes share no memory, no type system and no object model, so something has to translate. JInterface lets the JVM join the Erlang cluster as a *hidden node*: it speaks the same distribution protocol, it is addressable by node name, and it exchanges the same terms the Erlang side already sends.

Towards browsers and charge points: WebSocket. A charging page is open for the length of a charge, and the power allocated to a car changes every few seconds. The server has to speak first, which HTTP request-response does not allow, and polling at the frequency the data changes would be both late and expensive. These sockets do not end at Tomcat: Cowboy, the Erlang web server, runs inside the station node and pushes each new charging state straight to the browser, where a script renders it into the page Tomcat had served.

4.1.2 Server-side rendering, with real time only where it moves

The web front end is servlets plus JSP. A servlet gathers what a page needs, a JSP turns it into HTML, and what leaves Tomcat is a finished document. The browser draws it and keeps no application state of its own: every fresh view is a fresh request.

What that costs is the ability to push. A page is produced when it is asked for, so a value that changes on its own is seen again only on a reload. Most pages lose nothing by it, since they change when a person does something. Two views are different: the station page and the live session, where the allocated power and the delivered energy move every few seconds while the driver is watching. Those two open the WebSocket of section 9.2, and the script writes the moving values into a page the server had rendered.

4.1.3 Coordination separated from the sites

Three coordinator nodes hold the network-wide decisions. Each station node owns its connectors and its electrical budget, and it keeps working when the coordinators are unreachable.

This mirrors how the real industry deploys. Dynamic load management runs on a site controller physically installed at the charging site, because it must react in milliseconds to a car that starts drawing current and must keep the site alive when the link to the operator's cloud is down. What stays central is what can afford a network hop: the registry, the accounts, the invoices. Our station node is that site controller, and the requirement that a site keeps charging while isolated follows from the deployment being realistic in the first place.

The alternative was one central service holding everything, with the stations reduced to hardware adapters. It is simpler to write and it fails badly: an operator whose link goes down would stop delivering power to cars that are physically present and already plugged in. Section 8.3 reports what our split does instead, measured on 3 September with a station cut off from the cluster.

4.1.4 Volatile state in memory, durable state in MySQL

Two kinds of state, in two places, on purpose. Coordination state (who holds which connector, what each session is drawing, how far a battery has come) lives in the state of the Erlang processes that own it: it changes several times a second per connector, it is worthless once the thing it describes is over, and the hardware can rebuild it on reconnection. Durable state (accounts, vehicles, completed sessions, invoices) lives in MySQL, which is what the back office reads to draw its pages.

Nothing on the interactive path (granting a claim, allocating power) reads the database, so a database that is slow or briefly away does not stop cars from charging. The exception is

Table 3: The seven nodes of the delivered deployment.

Node	Technology	Responsibility
Back office	Java, Tomcat, JSP	accounts, authentication, station directory, history, billing; issues the JWT the station verifies
Coordinator ×3	Erlang/OTP	claims, cluster map, node monitoring, election and quorum; feeds the back office
Station ×2	Erlang/OTP, Cowboy	one process per connector, power allocation, the driver and charge-point WebSocket endpoints
MySQL	MySQL 8.0	durable state

authorising a *new* session, which has to resolve a vehicle to its owner, and section 8.3 describes what that costs an isolated site.

4.1.5 Where the system stops, and what an emulator stands in for

What we built is the coordination backend and its clients, which in industry terms is a charging-station management system together with its site controllers. The outlet, its contactor, its energy meter and the car attached to it are emulated.

The boundary is placed where a real interface already exists. Charge points talk to management systems over OCPP [1], an open protocol carried on WebSocket with JSON payloads, whose message set covers status notifications, meter values, reservations with an expiry, and charging profiles that cap the power of a connector. Our charge-point channel implements a **reduced message set modelled on OCPP 1.6-J**.

4.2 Architectural components

The delivered deployment is **seven nodes**, each a Docker container on one host. Table 3 lists them. Figure 2 places the seven nodes on the one network they share.

Back office (Java, Tomcat). Renders every page server-side and owns everything durable. It authenticates the driver, mints the JWT that the station will verify on its own, prices closed sessions, and applies the no-show suspensions. It also joins the Erlang cluster as a hidden node through JInterface. The leader pushes the station list to it whenever something visible changes, and at least every 30 seconds.

Coordinator (Erlang/OTP, three instances). One of them leads and the other two stand by. The leader grants and refuses claims and keeps the map of which stations are up; how the other two take over when it goes is section 7.4.

Station (Erlang/OTP with Cowboy, two instances). Each station runs one `gen_statem` per connector, a manager that arbitrates them and allocates the site power, and two WebSocket endpoints, one for drivers and one for charge points. The two stations of the delivered deployment are configured differently:

- `station1`: four connectors rated 150, 150, 150 and 50 kW, on a site budget of 350 kW. The demonstration profile lowers that budget to 200 kW so that the sharing is visible with two cars.
- `station2`: three connectors rated 150, 50 and 50 kW, on 180 kW.

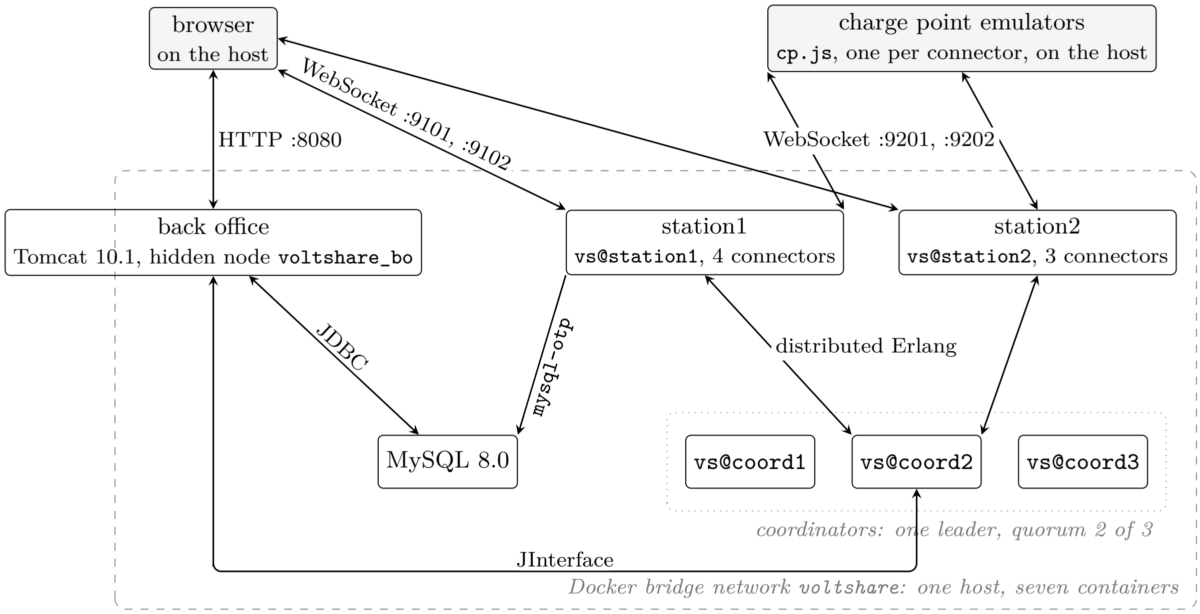

Figure 2: The delivered deployment: seven containers on one Docker bridge network, and the protocol on each edge. Both stations write to MySQL through `mysql-otp`; one edge is drawn. The browser and the emulated hardware run on the host and reach the containers through published ports.

Both sites are oversubscribed like in a real scenario. Four connectors that can draw 500 kW between them sit behind a 350 kW connection, so the moment three fast chargers are busy the station has to apportion rather than serve, which is P5 in section 7.5.

MySQL. One instance, holding accounts, vehicles, reservations that completed, sessions, invoices and notifications. It is a single point of failure for durable data and we declare it as one in section 8.8.

The browser is not a node. Pages arrive rendered, and the only state a browser holds is what its WebSocket has just pushed into the two live views so there is no client-side cache to reconcile after a reconnection, so a page that comes back gets a complete snapshot and is immediately correct.

5 Synchronization and Coordination Problems

This section states the problems the project set out to solve. The mechanisms that answer them are in section 7, and the ones that answer failures are in section 8.

Six of them are numbered P1 to P6 and are referred to by those names throughout the document.

5.1 The two exclusions in the system

Two resources in this system are exclusive:

A connector belongs to exactly one station, so the station decides alone. The vehicle instead, could appear at two stations at the same time, since the driver could open two browser tabs and presses *reserve* on both, each station finds its own connector free and grants, and neither can detect the breach, because the fact that would reveal it is held by neither. The second exclusion is the reason this project has a coordination problem at all.

Table 4: The two exclusions.

	Object protected	Contended by	Scope
P1	the connector	drivers, among themselves	inside one station
P2	the vehicle	stations, among themselves	across nodes

5.2 Problems inside one station

P1: several drivers want the same connector. Two requests for connector 3 arrive in the same millisecond. Both read the connector as free, both proceed, and the outlet ends up promised twice. This is the classic race on a shared resource, and its solution is section 7.1.

P5: the site cannot power everything at once. The sum of what the connectors can deliver exceeds what the site's grid connection can supply, which is true of real installations and is the reason dynamic load management exists.

A connector is exclusive and is therefore *granted*. Electrical capacity is divisible and is therefore *apportioned*, with the apportionment changing every time a car arrives, leaves, or slows down as its battery fills. See section 7.5.

5.3 Problems between the client and the system

P3: somebody stops answering. A driver reserves a connector and never arrives, or disappears in the middle of a session with the cable still in the socket. The same problem in a second form: a station holds a claim at the coordinator and dies, and the coordinator has no way to ask it whether the claim still matters. See section 7.3.

P6: everyone must see the same connector. Two drivers, the charge point and the back office all display a connector whose state changes several times a minute. The station pushes to all of them (section 4.1.2); what is left to decide is what it pushes. Sending only the change is smaller, but it leaves a client that misses one message wrong from then on, with nothing to tell it so. See section 7.6.

5.4 Problems across nodes

P2: one vehicle, one reservation, network-wide. The scenario of section 5.1: the same car reserving at two stations at once, with each station locally correct. See section 7.2.

P2b: the authority itself fails. Once P2 is answered by an authority, that authority becomes the thing that can fail. If it crashes, nobody can reserve until something replaces it. If a partition splits the network, both sides may conclude that the other is gone and each appoint its own authority, and two authorities granting claims for the same vehicle break exactly the guarantee the authority exists to provide. "Split brain" is worse than downtime here, because downtime refuses and split brain grants. See section 7.4.

P4: a station node goes away. It can crash, or its uplink can be cut.

A crashed node takes its sessions with it: the process metering them is gone and no row reaches the database. A partitioned node keeps delivering power to the cars plugged into it while the rest of the network stops hearing from it, so the site works and the operator goes blind. In both cases the vehicles that station was holding are stranded: their claims are alive in the coordinator, and their drivers cannot reserve anywhere else until those claims expire. See section 8.3.

6 Architecture Protocols

Table 5 maps the edges of the system. The middleware choices are argued in section 4.1.1 and the messages are specified in section 9. This section adds three things: which party opens each connection, which way authority flows, and what was kept off an edge.

Table 5: The edges of the system. "Opened by" is the side that dials.

Edge	Protocol	Opened by	Carries
Client-facing			
browser ↔ back office	HTTP	browser	forms and server-rendered pages; login session in `HttpSession`
browser ↔ station	WebSocket, JSON	browser	reserve, cancel, stop; the station snapshot and the live session, pushed
charge point ↔ station	WebSocket, JSON	charge point	cable and meter reports up, power limits and stop commands down; reduced OCPP 1.6-J
Internal			
station ↔ coordinator	distributed Erlang	station	claims, renewals, announcements, statistics; node monitoring in both directions
back office ↔ coordinator	JInterface	back office	the station directory and the leader, pushed; suspensions, pushed back
station, back office → MySQL	SQL	each	one `INSERT` per closed session from the station (`mysql-otp`); everything else from Java (JDBC)

6.1 Client-facing edges

The browser talks to two servers. That is the first decision on this map. Pages come from the back office over HTTP, rendered by servlets and JSP (section 4.1.2). Everything that happens at a station goes over a WebSocket opened *directly to the station node*: reserving a connector, watching a charge, stopping it.

The alternative was to route driver traffic through Tomcat, which would have kept one origin and one authentication scheme. We rejected it, because it puts the web tier on the path of every reservation and every live frame. A Tomcat restart would then blank every charging page in the network, and no station could keep a site working while the back office is down. The price of the direct connection is a second authentication. The back office mints a JWT, and the station verifies it locally without ever calling back (section 9.1.1).

The charge point has an edge of its own, kept apart from the driver's. The driver asks for a connector, the hardware reports what is plugged into it, and matching the two is the station's job. The charge point dials the station, one socket per connector, the way real equipment dials an operator's backend from behind a site network. The station never dials hardware (section 9.3).

6.2 Internal edges

Between Erlang nodes the protocol is distributed Erlang. Both sides address each other by registered name, so nobody holds an address, and the same connection carries `nodedown` (section 8.2). Stations call, the coordinator answers. The coordinator is never on the path of a charge, so a coordination outage costs only new reservations (section 9.4).

The back office joins the cluster as a hidden Erlang node through JInterface: it receives the directory and the leader as they change, and it pushes suspensions back. No page ever asks the cluster a question at request time (section 9.5). A hidden node takes no part in the quorum, and the web tier must never influence who leads.

MySQL is reached from two runtimes. The rule that makes that safe is written at the top of `schema.sql`: **one writer per table**. The station writes one row, the closed session, and Java owns the other tables and the price of a session after the fact. The no-show counter is written by Java alone, from a report the station sends through the coordinator, because a counter incremented from several nodes would have been a second contended object.

Two things are deliberately absent from the internal edges: a replicated log between the coordinators (section 8.4), and any link from one station to another.

7 Addressing the Problems

Section 5 listed the problems. This section describes the mechanism chosen for each one, the alternatives that were set aside, and what each choice costs.

7.1 Mutual exclusion on a connector (P1)

Each physical connector is owned by one Erlang process, a `gen_statem` implemented in `vs_connector`. Every request that touches that outlet becomes a message in that process's mailbox, and messages are handled one at a time, so two drivers reserving the same connector in the same millisecond are serialised by the runtime and the second finds it already `held`.

Which driver wins. Since Erlang guarantees FIFO order point to point and nothing across senders, the connector goes to the driver whose message the runtime delivers first. Mutual exclusion is the property claimed here, and nothing in this domain asks for more.

The machine gives a second guarantee: with the wrong vehicle, `charging` is unreachable from `held`, so no session can start on a connector somebody else holds. The transition does not exist, so there is no check to forget.

7.2 One reservation per vehicle, network-wide (P2)

The mechanism adopted is a *claim*: a permission, granted by the coordinator, recording that a given vehicle is now committed and to which station.

7.2.1 Claim first, commit after

A station obtains the claim *before* it creates the reservation.

In the wrong order, a crash between the two steps leaves a reservation that no other node knows about, allowing the vehicle can then obtain a second reservation elsewhere. In the right order, the same crash leaves a claim that was granted and never used. Nobody can reserve for that vehicle for a few minutes, and then the claim expires on its own. The system errs towards refusing rather than towards granting twice.

7.2.2 Why a central arbiter

The coordinator is a single process, `vs_coord_srv`, and its claim table is a map keyed by vehicle. Requests from different stations queue in its mailbox and are decided one at a time. This is the centralised mutual exclusion where there is one critical section per vehicle. Two claims for two different cars contend over nothing, and what the mailbox serialises is the instant of the decision.

Three designs were considered.

One coordinator, unreplicated. The simplest, and it leaves an avoidable single point of failure.

A `UNIQUE` constraint in MySQL. Correct, and almost free to write. But the coordinator has to exist anyway for the cluster map and for node monitoring, so this removes no component, and a claim that *expires by itself* is a timer inside a process, while in a database it becomes a cleanup job or a filter on every read.

Direct coordination between stations (Ricart-Agrawala). No central authority, at $2(n-1)$ messages per reservation, and it serialises access to *one* critical section: here two reservations for two different vehicles would wait for each other with nothing to gain.

What we built is the first one with its single point of failure removed: one arbiter serving at a time, elected among three coordinator nodes and required to hold a quorum, which is section 7.4.

7.2.3 Ordering decided by one clock

When a claim is granted, the coordinator stamps it with `granted_at` and sends that timestamp back with the grant. The station stores it and echoes it in the renewal it sends every ten seconds for all the claims it holds (section 7.3).

The purpose is conflict resolution after a failover. If two claims for the same vehicle ever meet in the table, the older one wins, and the comparison is between two timestamps produced by the *same* process. No two machines with unsynchronised clocks are ever compared.

7.3 Leases (P3)

The problem of the driver who reserves and never arrives is approached by a lease: every reservation is granted with an expiry, and when it passes the connector returns to `free` with no operator action and no cooperation from the party that disappeared.

There are two nested expiries, and they protect against two different absent parties:

- the **reservation lease** protects against the driver who never shows up. It is the timer the connector process holds;
- the **claim expiry** protects against the station that dies without releasing what it holds. A renewal keeps the claim alive, so a station that crashes stops renewing and its claims expire on their own.

7.4 Coordinator replication: election and quorum (P2b)

Three coordinator nodes run, `vs@coord1`, `vs@coord2` and `vs@coord3`, and every node of the cluster is configured with the same three names. One of them is the leader and is the only node that grants or refuses claims. A station that addresses one of the other two is redirected to the leader.

Election. When the leader stops responding, the survivors elect a new one with the bully algorithm over those names: the list is sorted, and the highest live entry wins, so a cluster in good health is led by `vs@coord3`. A node that notices the leader is gone sends `ELECTION` to the nodes above it. Silence means it is the most senior node alive and it announces `LEADER` downwards; an answer means somebody more senior is handling it, and it waits. With three nodes this settles in two rounds of messages. Detection is by timeout, which means we assume a *synchronous* system, an upper bound on response time.

Quorum. Bully alone does not prevent split brain. In a partition each side happily elects its own local maximum, and two authorities granting claims break precisely the invariant the coordinator exists to protect. So a coordinator serves only while it can see a majority of the configured coordinator cluster, itself included: two out of three. A node in the minority suspends itself and refuses everything, leader or not.

This is why the cluster has three members and not two. With an even number, a partition can produce two halves that each consider themselves entitled to grant.

State after a failover. Since a new leader inherits nothing, it asks the stations what they hold and rebuilds the claim table from their answers. The reason of this approach is developed in section 8.4.

7.5 Sharing the site power (P5)

An Erlang module, `vs_power`, recomputes the split on every arrival and departure, and on a periodic tick that follows the charging curve as batteries fill.

The policy is max-min fair share with hand-back. Every active session starts from $budget/N$. A session that cannot absorb its share, either because the car is small or because it is tapering near full, takes only what it asks for and hands the surplus back, which is then split again among the others. The loop repeats until nobody hands anything back or the budget is exhausted.

7.6 One source of truth for every client (P6)

The cheapest way to make every client agree is to give them nothing to compute. The station holds the authoritative state and pushes a *complete* snapshot of it, so that each page that misses a message is wrong only until the next push, and one that reconnects is correct immediately. Snapshots go out on every change and on a periodic tick.

8 Fault Tolerance and Recovery

Fault tolerance in this system relies on five mechanisms selected based on the type of fault.

8.1 The failure model

We assume crash-stop and network partition. Byzantine behaviour is out of scope. Table 6 lists what can fail and what the system does about it.

8.2 Two failure detectors, because there are two failures

A coordinator that crashes and one that is cut off look different on the wire. A crash closes the socket, and `net_kernel:monitor_nodes/1` reports it at once. A partition closes nothing: the peer is unreachable while its connection stays open, so nothing arrives to say so, and a leader in the minority would go on granting claims believing it still had its quorum.

Liveness is therefore decided by a heartbeat among the coordinators, implemented in `vs_coord_membership`: each announces itself once a second, and three missed announcements mean gone. A node that finds itself below a majority suspends itself and stops serving, leader

Table 6: What can fail, how it is noticed, and what it costs.

What fails	How it is noticed	What happens	What is lost
Coordinator, crash	`nodedown`, at once	election; the new leader rebuilds from the stations	nothing
Coordinator, partition	heartbeat, 3 s	the minority suspends itself	new reservations, minority side
Station node, crash	`monitor_node`, at once	the coordinator drops that station's claims	sessions running there
Station uplink, partition	`monitor_node`, on the tick	the site keeps charging, starts nothing new	the operator's view
Charge point	socket closes, then 30 s grace	session closed with the last measured energy	nothing measured
Back office	nothing, it is a hidden node	the cluster is unaffected; it re-reads on restart	nothing durable
MySQL	connection errors	not tolerated, see section 8.8	writes, until it returns

or not. The heartbeat runs among the three coordinators only, because that is where a stale view produces two leaders; a station that goes silent is noticed later, at the cost table 6 states.

8.3 A station dies

`vs_coord_srv` monitors every station it has heard from and, on `nodedown`, erases that station's claims. Keeping them would shut those drivers out of the whole network until the lease expired.

The cars carry on charging, because the node is not on the path that delivers power. The charge point is also the side that counts the energy, so when the station comes back it reports its running total, the station adopts the session from it, and the charge is billed in full. What is lost is a car that unplugs while the node is still down: nothing writes its row, and that energy is never invoiced.

8.4 Rebuilding the claim table after a failover

The new leader must discover the claims by querying the system. It sends a `who_do_you_hold` message to each station; the station examines its connectors and responds by indicating those in the `held` (occupied) or `charging` state, based on its local memory. The leader transitions from the `rebuilding` state to the `serving` (operational) state once the responses are received. Requests received in the interim are rejected with a retry indication, which the stations interpret as a temporary refusal.

Convergence via two paths. The system would converge autonomously without the need for explicit queries from the new coordinator, as stations renew their claims every 10 seconds; however, the query allows for a significant reduction in recovery time.

8.5 A charge point dies

A lost socket is not a broken charger, so the connector waits ninety seconds before doing anything: sixty of socket idle timeout, thirty of its own grace. If they run out mid-session, the session is closed with the last energy the charge point reported, the only figure anybody measured, and the connector goes `out_of_service` until a charge point boots and reports itself available.

8.6 Supervision inside a node

Three supervisory strategies have been implemented:

```
vs_station_sup                      one_for_one
  +-- vs_connector_sup              simple_one_for_one
  |     +-- vs_connector            connector 1   started at boot by
  |     +-- vs_connector            connector 2   vs_station_mgr, one
  |     +-- vs_connector            connector N   per physical connector
  +-- vs_station_mgr
  +-- vs_claim_client
  +-- vs_station_db

vs_coord_sup                        rest_for_one
  +-- vs_coord_srv                  claim table, cluster map
  +-- vs_coord_bo                   JInterface bridge to the back office
  +-- vs_coord_election             bully election
  +-- vs_coord_membership           heartbeat and quorum
      declaration order is dependency order
```

Listing 1: The two supervision trees. The station nests a second supervisor inside the first; the coordinator does not.

A supervisor restarts its children, and the strategy decides which ones go down together. The two trees choose differently.

The coordinator is `rest_for_one`: its children are declared in dependency order, so a process that dies takes with it everyone declared after it. The bridge publishes what the claim table holds, and must not outlive a table that was restarted underneath it.

The station is `one_for_one`: its children are independent and restart alone. Connectors find the manager by registered name and hold no reference to it, so a manager that comes back re-adopts them and no session is lost. The connectors are all the same kind of process, one per outlet, hence `simple_one_for_one`; one that crashes comes back in `free`.

8.7 Delivery semantics, chosen per message

The bridge from the coordinator to the back office carries three kinds of event, and they deliberately do not get the same guarantee, because the cost of a duplicate is different for each.

- **The session row: at-least-once over an idempotent write.** The message itself is never resent. What makes it at-least-once is that the station has already written the row at the end of the session, and the back office sweeps `cost_cents IS NULL` every sixty seconds, so the event only makes an invoice arrive sooner and losing it costs one interval. The price is written with `UPDATE sessions SET cost_cents = ?  WHERE id = ?  AND cost_cents IS NULL`: a repeat updates zero rows, which is not an error.
- **The no-show strike: maybe.** One send, no retransmission and no duplicate filter. A counter cannot recognise a duplicate, so a retry could suspend an innocent driver, and a lost strike is a smaller wrong than an invented one.
- **The notification to the driver: maybe**, for the same reason. A duplicate is a row somebody reads twice, a loss is a row they never see, and we chose the quieter failure.

8.8 What tolerates nothing, and is declared

Three things here have no recovery path.

MySQL is a single point of failure for durable data. Replicating it was out of reach for this project. This choice leads some limitations:

Sessions on a dead station are lost, as section 8.3 describes. The process that was metering them is gone.

The rebuild of the claim table can still miss a station that is unreachable at exactly the wrong moment. Its claims are absent from the new leader's table, so one of its vehicles could obtain a second claim before the conflict is noticed. The window is narrow and it is real, and it closes when the station returns and re-presents the older claim, which wins.

9 API and Protocol Specification

Five channels carry everything in this system, and section 6 said which runs where and why. This section specifies what travels on each one.

9.1 HTTP: browser to back office

Nine servlets, mapped with `@WebServlet` annotations. Everything is rendered server-side and forwarded to a JSP; there is no JSON API on this channel, because the browser receives pages.

Table 7: HTTP endpoints of the back office. The last column marks the ones behind `AuthFilter`.

Path	Method	Effect	Auth
`/register`	POST	creates the account and its vehicle profile	
`/login`	POST	authenticates, opens the `HttpSession`, mints the JWT	
`/logout`	GET	invalidates the session	
`/stations`	GET	the directory, read from `StationDirectory` in memory	•
`/station`	GET	one station's page, with the JWT and the WebSocket URL	•
`/session`	GET	the live session page for a connector	•
`/profile`	GET	account and vehicle	•
`/history`	GET	past sessions with their invoices	•
`/notifications`	GET	notifications stored for this driver	•

9.1.1 The token the station will trust

A station has to recognise a driver without asking the back office, which is what keeps a site working while Tomcat is down. The token is the only thing the two sides share.

It is a JWT signed with **HS256** using a symmetric secret, passed to both sides in the environment as `VOLTSHARE_JWT_SECRET`. It lives 60 minutes. Java signs with `jjwt`, Erlang verifies with `jose`.

```
{ "sub": "12", "username": "andrea", "vehicle_id": 88,
  "iss": "voltshare-backoffice", "exp": 1755792000 }
```

Listing 2: The claims, plus the standard `iat`.

`sub` is the user id as a string, which the JWT specification requires, and `vehicle_id` is a number. The station needs both: the first attributes the session to an account, the second matches the claim, which is per vehicle.

On the first frame of the WebSocket the station verifies the signature, checks that `iss` is exactly `voltshare-backoffice`, checks that `exp` is in the future, and extracts the two identifiers. A failure closes the socket with `4401`, an expired token with `4408`. The station never calls the back office about a token, whether it accepts one or throws it out.

How the token reaches the browser. The servlet puts it in the session, the JSP renders it into a hidden `data-` attribute through `<c:out>`, and the WebSocket code reads it from there.

9.2 WebSocket: driver to station

Messages are JSON objects, and both directions share one envelope: a client frame names an `action` and a `request_id`, a station reply carries a `type` and echoes the `request_id` it answers, and an unsolicited push reuses the same shape with a null request id.

What the client sends.

- `join` is the first frame and carries the JWT of section 9.1.1, within 5 s. The station verifies it locally and binds the user and vehicle ids to the connection. Nothing else is accepted until it succeeds, and a `user_id` in any later payload is ignored.
- `reserve` names a connector. The station asks the coordinator for a claim *before* it touches the connector (section 9.4); only a granted claim moves the connector to `held`, and the reply carries the lease the page counts down from, given both as an instant and as a duration on the station's clock.
- `cancel_reservation` frees the connector and releases the claim at once, with no penalty, so a driver who will not come has no reason to keep holding it.
- `stop_session` tells the charge point to stop delivering and returns the connector's share of the site power to the pool. The `sessions` row is not written until the vehicle is unplugged, so time still plugged in after the charge can be billed as overstay.

A refused action comes back as an `error` frame that echoes the `request_id` and carries a code the page shows to the driver: `ALREADY_HELD` when another driver holds the connector, `NO_CLAIM` when the vehicle is reserved elsewhere or the coordinator never answered, `SUSPENDED` for an account serving a no-show penalty, `INVALID_STATE` when the action does not fit the connector as it is now, and four more in the same spirit. The connection stays open. A protocol fault instead closes the socket, with separate close codes for a bad token, an expired token, and a wrong `station_id`, so the page can tell an expired login from a transient fault.

Figure 3 puts these frames on a timeline, with a lost `ack` and a second reservation for the same vehicle.

What the station pushes. An `ack` or an `error` answers each action. Three more frames arrive unasked.

- `state` is the whole station: its power figures, whether the coordinator is reachable, and every connector with its state and, for the caller, whether that reservation or session is theirs. Sent on join, on every change, and every 5 s. A connector's state is one of `free`, `held`, `charging`, `suspended`, `complete`, `overstay`, `closing` or `out_of_service`.
- `session` reaches only the driver of a running session, and carries its live figures: power, energy, state of charge, time to full, overstay seconds net of the grace period and the last one is marked `closed`.
- `notification` is a one-off message for the driver it names, in six kinds: `reservation_expiring`, `reservation_expired`, `claim_revoked`, `charge_complete`, `overstay_started` and `session_interrupted`.

Decisions on this channel.

- **L'identità è determinata esclusivamente dal token.** L'appartenenza di una prenotazione o di una sessione al chiamante viene stabilita dalla stazione in base all'ID

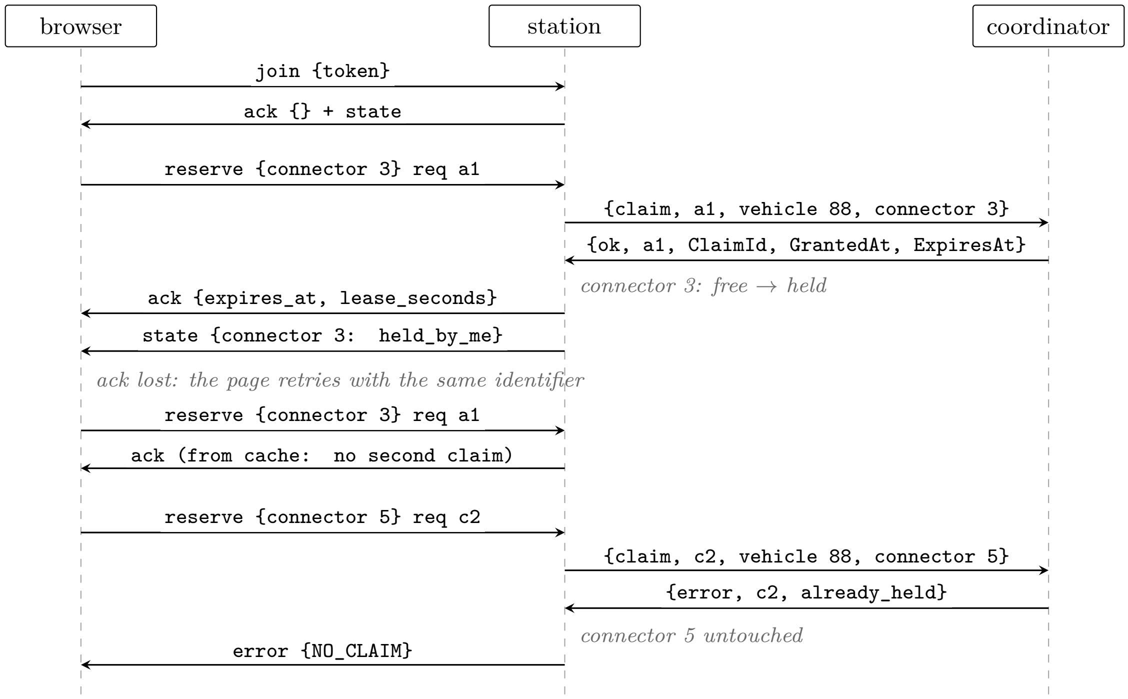

Figure 3: A reservation on the driver channel: the claim precedes the commit, a retried `reserve` is answered from the cache, and a second reservation for the same vehicle is refused by the coordinator before the station touches the connector.

associato al token. Includere un campo `user_id` nella richiesta permetterebbe a un driver di effettuare o annullare prenotazioni per conto di terzi semplicemente modificando un frame; per questo motivo, la richiesta ne è priva.

- **Un unico codice `RETRY_LATER`, con la causa specificata nel messaggio.** Tale codice copre tre condizioni transitorie (ricostruzione del coordinatore, avvio della stazione, processo del connettore in fase di riavvio dopo un crash).

9.3 WebSocket: charge point to station

This channel is the boundary of the system. Above it is production logic, below it is emulated hardware. The message set is a reduced OCPP 1.6-J [1].

The charge point is the client and the envelope is the one of the driver channel (section 9.2). One socket per connector, so the socket *is* the connector and nothing here has to be routed. Nothing is retried on this channel and there are no `error` frames, so a frame the station cannot read is logged and dropped. There is no authentication either. The link is site-local.

Eight messages, and a connector's whole life is one sequence of them. The equipment connects and says `boot`, then a `heartbeat` every 30 s and a `status` at every change of physical condition. `plugged` when a cable goes into a car, a `meter` every 5 s while it charges, `unplugged` when the cable comes out. Two frames travel the other way, and the equipment obeys both without answering.

What the charge point sends.

- `boot` binds the socket to the connector. Only the physical `status` of its four fields has an effect: the rating that governs the allocation comes from the station's own record, so what the equipment claims about itself ends up in one line of the log. The `ack` is the whole configuration of a charge point, which therefore carries none of its own: the two

Table 8: The message set: what each frame says, and what it carries.

Message	Dir.	What it says	Payload
`boot`	→	the equipment is on line, and this is what sits behind the connector	vendor, model, firmware, `rated_kw`, physical `status`
`heartbeat`	→	the equipment is still alive	empty
`status`	→	the physical condition of the plug has changed	`available`, `occupied`, `faulted` or `unavailable`
`plugged`	→	a cable has gone into a car, and this is which car	`vehicle_id`, `soc_pct`, `battery_kwh`, `max_kw`
`meter`	→	this is the power flowing now and the energy counted so far	`power_kw`, cumulative `energy_kwh`, `soc_pct`
`unplugged`	→	the cable has come out, and this is the final total	`energy_kwh`
`set_limit`	←	this connector must not draw more than this	`limit_kw`
`stop`	←	end the delivery now, for this reason	`reason`, one of six

intervals, the station's clock, and the limit in force. A connector that has no process behind it at that instant, because the node is starting or its supervisor is restarting it, is admitted at the handshake and refused here. `connector not ready` is temporary and the equipment retries on its own backoff; `unknown connector` is permanent.

- `heartbeat` and `boot` are the only messages that get an `ack`.
- `status` outside the four values is logged and dropped.
- `plugged` is the one place where authorisation happens: the station resolves the vehicle to an account locally, which is the one lookup an isolated station cannot make (section 8.3). `vehicle_id` and `max_kw` are mandatory and a payload missing either one is refused. Without the first there is nobody to bill; without the second the allocator of section 7.5 would compute a demand of zero and leave the car at zero kilowatts for ever, with nothing in any log to say why. The other two only sharpen the estimated time of arrival.
- `meter` counts energy cumulatively since the session started, and the stored total is never allowed to go down, so a counter that resets on a firmware glitch cannot subtract energy already billed. `power_kw` sits below the limit whenever the car itself is tapering, and that gap goes back to the other connectors. A reading with no session behind it is dropped and logged.
- `unplugged` writes the `sessions` row with energy and overstay, releases the claim, and moves the connector to `free`.

What a `plugged` does next depends on the connector it arrives at.

What the station sends back. Two commands travel down, and both carry a null `request_id` because the station starts them.

- `set_limit` is the allocator's output, a single instantaneous ceiling in kW. A limit of zero is the wire's word for suspended: the session stays open and draws nothing.
- `stop` ends the delivery, with one of six reasons: `driver_stopped`, `target_reached`, `claim_revoked`, `not_your_reservation`, `faulted` and `station_shutdown`. The last goes out just before the closing frame, because once the station is down there is nobody left to tell. After a `stop` the connector stays occupied until the cable comes out, which is what makes an overstay billable. `faulted` is the exception: the session is closed at once with the energy last measured, because faulted equipment may never send another frame.

Table 9: What a `plugged` does, by connector state.

Connector	Vehicle	Outcome
`held`	the reserved one	session starts, lease cancelled, claim kept to the end
`held`	another	`stop` with `not_your_reservation`; the reservation stays
`free`	any	walk-in: session on that vehicle's account, no claim needed
`free`	of a suspended account	session starts anyway: the penalty covers reservations only
`charging`, `complete`, `closing`	any	refused with no frame; physical and logical state have diverged, and the log says so

Silence. The contract obliges the equipment to speak, so the missed-heartbeat rule is the idle timeout on the socket and this channel needs no `ping`. The budget is three heartbeats, 90 s, and the part worth knowing is that it is spent in series and not in parallel. The listener waits 60 s, two intervals, and closes the socket. Only then does the connector see the socket die and arm its own 30 s of grace. Adding the two gives the 90 s.

Reconciliation after a station restart. The connector processes die with the node, so the station has no memory of the session. The charge point reconnects, boots, and re-sends `plugged` with two optional fields: the cumulative energy it has delivered and the seconds it has been charging. The station opens a session from those figures instead of stopping a car that is charging happily. `started_at` is *now* minus `charging_seconds`, read on the station's own clock, so two machines whose clocks are minutes apart still produce the same instant. The reservation is not reconstructed. In the mirror case, where the socket survives and only the connector process died, the replay refreshes the energy from the last `meter` and leaves `charging_seconds` out, because a duration measured earlier no longer says how long the charge has been running.

Decisions on this channel.

- **Limits are repeated on every tick**, even when unchanged, so a missed frame is corrected within one tick.
- **The station keeps the clock.** What the charge point reports is a duration: the seconds it has been charging. The station subtracts it from its own current time, and that is `started_at`. An instant sent by the hardware was rejected, because nothing keeps the two clocks together.

9.4 Claims: station to coordinator

Distributed Erlang, with the coordinator a `gen_server` registered locally as `vs_coord_srv` on each coordinator node and addressed as `{vs_coord_srv, Node}`. Calls carry a 2000 ms timeout, and a timeout is treated like an unreachable node.

9.4.1 Acquire claim

```
{claim, ReqId, VehicleId, UserId, StationId, ConnId}

{ok, ReqId, ClaimId, GrantedAt, ExpiresAt}
```

Table 10: The claim protocol. `ReqId` is generated by the station and echoed back.

Message	Direction	Purpose
`claim`	station → coord	ask before committing a reservation
`renew`	station → coord	re-present every claim held, every 10 s
`release`	station → coord	fire and forget, on cancel, expiry or completion
`who_do_you_hold`	coord → station	the rebuild query of section 8.4
`station_up`	station → coord	announcement, at start-up and every 30 s
`station_stats`	station → coord	free, held and charging counters
`session_closed`	station → coord	a session row was written, relay it to Java

```
{error, ReqId, already_held | suspended | rebuilding | unknown_station}
{not_serving, LeaderNode | undefined}
```

Listing 3: Request and the four possible replies.

Each refusal means something different to the driver, so the station maps them apart: `already_held` says this car is committed at another station, `suspended` says the account is serving a no-show penalty, and `rebuilding` is temporary and produces a retry rather than an error on screen. `unknown_station` means the coordinator has not heard the announcement yet, so the station re-announces itself and tries once more.

9.4.2 Renew

A station renews all its claims in one message every ten seconds. The renewal is what keeps them alive: a claim is a lease, and one that stops being renewed expires on its own, which is how a station that dies stops holding vehicles it can no longer serve. The reply names those still valid and those revoked.

```
{renew, StationId, [{ClaimId, VehicleId, ConnId, UserId, GrantedAt}]}
{renewed, Ok, Revoked, NewExpiresAt}
```

Every renewal carries `GrantedAt` and `UserId`, including the hundredth one for a claim nothing has disturbed. The reason is recovery: a renewal is how a newly elected leader learns about claims it never granted. A claim it has never seen is **accepted** and adopted with the timestamp the station reports, which is what makes section 8.4 converge even when the rebuild query reaches nobody.

9.4.3 Finding the leader

The station keeps its own record of the current leader in `vs_claim_client`. If the coordinator queried is not the leader, it responds to the station's claim message with a `not_serving`. This coordinator replies with the new leader's name if known, and the call is retried once. A `not_serving` message lacking this information, a timeout, or a `noproc` error causes the station to iterate through the configured list using a round-robin approach for up to one full cycle to locate the new leader. This situation can arise following a network partition between coordinators, causing an isolated coordinator to lose track of the current leader.

If the attempt fails, the station gives up and rejects the reservation. It neither queues the request nor blocks the connector management process; this rule ensures that an interruption in the coordination service does not affect vehicles that are already charging.

9.5 The JInterface bridge: coordinator to back office

The back office does not query the cluster but joins it. Java opens an `OtpNode` named `voltshare_bo`, registers a mailbox called `backoffice`, and receives Erlang terms as the coordinator generates them.

Java acts as a **hidden node**: it participates in the distribution protocol and is addressable by name. The bridge runs on its own daemon thread and attempts to reconnect every three seconds; consequently, restarting the coordinator never requires restarting Tomcat.

Table 11: Messages on the bridge.

Message	Direction	Purpose
`stations_update`	coord → Java	the complete station list, on change and every 30 s
`leader`	coord → Java	who is serving now
`session_closed`	coord → Java	a session was written; price it now
`no_show`, `show_up`	coord → Java	penalty accounting
`notify`	coord → Java	something to tell a driver next time they look
`get_stations`	Java → coord	asked once on connect, so no page starts empty
`user_suspended`	Java → coord	a no-show counter tripped
`user_unsuspended`	Java → coord	the penalty expired

`stations_update` is always a complete snapshot. `StationDirectory` replaces the entire list atomically, as a partial update would leave a station on the page even after its node had gone offline.

Suspension recovery occurs via push. The set of suspensions resides in the coordinator's memory, while the associated counter is stored in MySQL; consequently, a newly started coordinator has no knowledge of them. Therefore, the back office re-pushes every active suspension upon receiving each `leader` announcement. A coordinator that restarts and resumes the role converges without requiring any explicit request.

Recovery runs in opposite directions on the two edges, and for two reasons. The stations own the claims and have to be up for the cluster to mean anything, so the leader asks them, and it cannot serve until they answer: granting without knowing the claims would break P2. Nobody in the cluster owns a suspension, which lives in the database behind the back office, and the back office is allowed to be down (table 6); a leader that had to ask it would be unable to start serving whenever Tomcat was down. A leader that does not yet know the suspensions serves anyway, and the only thing it gets wrong is failing to refuse an account it should have refused, which is why that one can arrive late.

What happens when the bridge fails. No coordinator reachable at start-up means retries every three seconds, and pages that show an empty list with a warning. If a coordinator crashes while Tomcat is running, the last snapshot remains in memory, marked as obsolete after 90 seconds (i.e., three missed updates). At that point, the warning is issued. If Tomcat dies, the cluster is left in a consistent state, as the back office is responsible for managing the reservations and sessions. Tomcat dying costs the cluster nothing, and costs nothing durable either. Every message towards the back office is a send to a mailbox that is no longer there, dropped without an error and without a retry, so the coordinator neither blocks nor notices; reservations and charging sessions never touch the back office in the first place. The session rows were written by the stations and are already in MySQL, so the first sweep after a restart prices everything that closed in the meantime. What is lost is the notifications raised during the outage, which have no row behind them.

10 Future Works

Three extensions, none started, each with the reason it was set aside.

A waiting list for a full station. A driver who finds every connector taken has nothing to join. The feature needs a per-station queue with an owner, and an offer that is itself a lease, so that a waiter who never comes back cannot freeze the queue.

Native push notifications. There is no mobile push. The overstay notice reaches a driver who is looking at the page, or who reopens the application later, so the grace period assumes an attentive driver, which is the one thing a driver who has walked away from the car is not. A mobile application would close that gap. The event to deliver already exists and the station already emits it, so what is missing is only the channel: the WebSocket of section 9.2 lives as long as the tab, while a push registration belongs to the device and outlives the browser.

An operator console. The back office is read-only. Taking a connector out of service by hand, or closing a session that a charge point has left hanging, would be a command for the operator dashboard.

Repo git: https://github.com/andreainno22/Progetto_DSMT

References

[1] Open Charge Alliance. *Open Charge Point Protocol JSON 1.6 (OCPP-J 1.6) Specification*, part of OCPP 1.6 edition 2, 2017. https://openchargealliance.org/protocols/ocpp-protocols/ocpp-1-6/.